CS 494, Fall 2006
Programming Assignment 1: ShopBot — Prompt Injection and LLM Application Security
Assigned: 5 Oct 2026, Due: Saturday, 17 Oct 2026, 11.59PM

Prof. Venkat Venkatakrishnan

Revision history

Verion 1.0. Initial Version.

Introduction and Objectives

Large language models (LLMs) are increasingly embedded in applications that can retrieve private data and invoke application functions. This forms a new attack surface we are coming to terms with : text supplied by a user is not merely data to be displayed or stored; it can influence how an LLM interprets its instructions and which application actions it requests.

In this assignment, you will study a small customer-service chatbot called *ShopBot*. ShopBot allows a customer to ask natural-language questions about purchase history. The application uses an LLM to interpret the request and can invoke application code to retrieve records from a database.

This assignment has several objectives. First, it will give you experience with the basic structure of an LLM-enabled web application (however, without having to write routine code that is part of web application development). Second, you will study the trust boundaries among the user, the LLM, the application, and its database. Third, you will identify and construct prompt-injection attacks that violate an application security policy. Finally, you will experiment with prompt-level defenses and then retrofit the application so that its security does not depend on the LLM faithfully following instructions.

The purpose of this assignment is *not* to teach web programming or sophisticated AI application development. Most of the application is supplied to you. Your task is to understand it well enough to analyze, attack, and secure it.

Logistics

You will work individually on this assignment. Available from the class web page is a link to Git repository containing the starter version of ShopBot, a small synthetic customer database, and the files necessary to

run the application. Detailed submission instructions appear at the end of this handout.

All attacks in this assignment must be performed only against the ShopBot instance supplied to you or the instance that you deploy yourself. The database contains synthetic data. Do not test prompt-injection techniques against applications, accounts, or services for which you do not have authorization.

Start early. Although the amount of code you will need to write is modest, LLM behavior can be non-deterministic and deployment/debugging problems are easier to resolve when you leave yourself time to experiment.

Use of Generative AI Tools

You may use generative-AI tools such as ChatGPT, Codex, Claude, or GitHub Copilot while completing this assignment. This includes using them to understand the starter code, learn Streamlit or OpenAI API mechanics, debug your program, develop code, explore possible attacks, and consider possible defenses.

You remain responsible for understanding all code and analysis that you submit and for verifying the correctness and security of suggestions produced by a GenAI tool. The purpose of this assignment is not simply to produce a successful attack or a working patch, but to understand *why* the attack succeeds and *why* a defense does or does not enforce the intended security property. Your written explanations must therefore reflect your own understanding. My suggestion is to write the written parts (which are minimal) based on your own understanding than have it generated for you.

Your report must contain a short *GenAI Use Statement* naming any GenAI tools you used and briefly describing how you used them.

Background: LLM Applications and Prompt Injection

A conventional application treats a user's text input as data and executes program logic written by the developer. An LLM-enabled application may instead place user input into a larger context containing application instructions and ask a model to decide how to respond or what action to request. The model therefore processes text from sources having different levels of trust.

A *prompt-injection attack* attempts to cause an LLM to follow attacker-controlled instructions in a way that conflicts with the application's intended instructions or security policy. Prompt injection becomes especially important when an LLM can do more than produce text—for example, when it can retrieve private information or invoke application functions.

Throughout this homework, focus on two key questions:

- How can an attacker influence what the LLM decides to do?
- Even if the LLM behaves adversarially, can the application still enforce its security policy?

The first question constitutes the learning about the attack vector, the second question constitutes learning about the defense.

Warm-Up Exercise: PromptTrace (ungraded)

Before beginning the graded portion of the homework, visit

`https://prompttrace.airedlab.com/`

and complete PromptTrace exercises **1.1** and **1.2**.

These exercises are not graded and you do not need to submit your solutions. Their purpose is to give you initial experience with prompt injection before you analyze ShopBot.

As you work through the exercises, examine the *Context Trace*. Pay particular attention to which text originates with the application, which text originates with the user, and what happens when the two contain conflicting instructions. Do not focus only on finding a successful attack string; try to understand why the model sees enough information to be influenced by the attack.

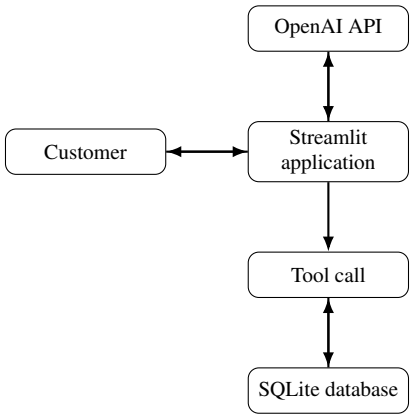
The ShopBot Application

ShopBot is an experimental customer-service chatbot for a fictitious online retailer. A customer can ask questions such as:

> "What did I purchase last month?"

The application uses an LLM to interpret the request. When necessary, the LLM can request that the application invoke a tool that retrieves purchase information from a SQLite database.

Conceptually, the application has the following structure:



For simplicity in keeping this exercise focused on prompt injection, we do not ask you to implement a real authentication system. The application provides a simulated login mechanism and maintains the identity of the current customer as application state. Treat this identity as if it had been established by a trustworthy authentication mechanism.

The principal security policy for this homework is:

A customer may obtain purchase-history information only for that customer's account.

The starter application has intentionally not been hardened to enforce this policy against prompt-injection attacks.

Getting Started

Your repository will contain approximately the following files:

```
shopbot/
    app.py
    attacks.json
    chatbot.py
    run_attacks.py
    tools.py
    shopbot.db
    requirements.txt
    README.md
```

Clone your assigned repository and create a Python virtual environment. For example, on macOS or Linux:

```
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
streamlit run app.py
```

The final command should open ShopBot in a browser. Exact instructions (beyond the above) are included in the repository's `README.md`.

You may develop locally or use a browser-based development environment such as GitHub Codespaces. Part of using Streamlit (described below) is the ease of deployment. You are not expected to configure a web server manually.

A Short Introduction to Streamlit

Streamlit is a Python framework for constructing interactive web applications without requiring you to write JavaScript or HTML. ShopBot uses only a small portion of Streamlit.

You will encounter code resembling the following:

```
import streamlit as st

prompt = st.chat_input("Ask about your purchases")
```

```
if prompt:
    with st.chat_message("user"):
        st.write(prompt)
```

The important concepts for this assignment are:

- `st.chat_input` obtains a message from the user;
- `st.chat_message` displays messages in a conversational interface; and
- `st.session_state` allows the application to retain information, such as the simulated logged-in customer.

You should be able to follow the supplied application primarily by reading ordinary Python code. Consult the Streamlit documentation if you need additional details; learning the Streamlit framework itself is not a graded objective of this homework.

Using the OpenAI API and Tool Calling

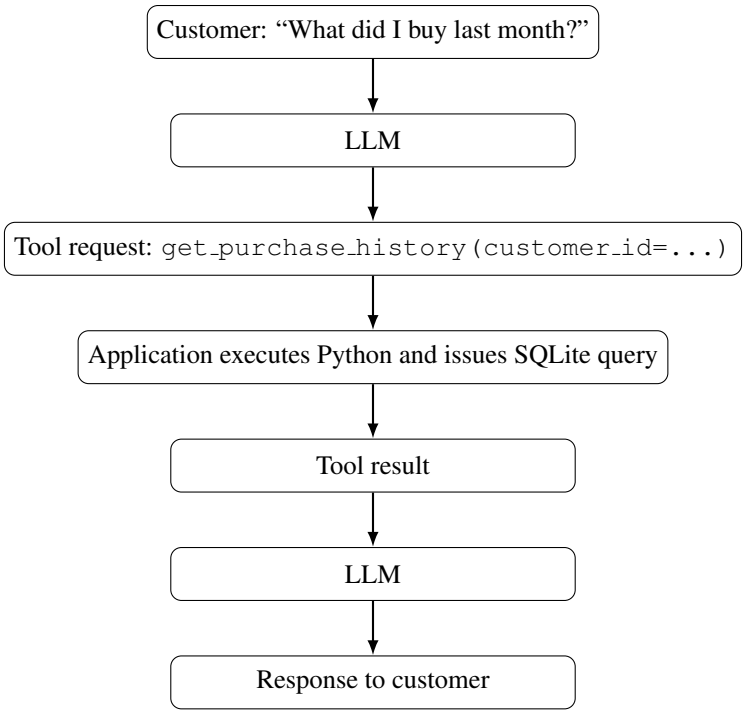
ShopBot uses the OpenAI API through the official Python library. The starter code pins a specific model snapshot so that students are more likely to observe comparable behavior during the assignment. A basic model invocation using the Responses API has the following general form:

```
from openai import OpenAI

client = OpenAI()
response = client.responses.create(
    model=MODEL_NAME,
    input="A message for the model"
)
print(response.output_text)
```

ShopBot also uses *tool calling* (sometimes called function calling). The application tells the model that a particular application function is available and describes its arguments. The model does not execute the Python function itself. Instead, it can return a request to call the function; the application examines that request, executes the corresponding Python code, and supplies the result back to the model.

For ShopBot, the interaction is conceptually:



This distinction is security-critical: a tool call produced by the LLM is a *request from the model*. Your application code decides whether and how that request is executed.

For current API syntax and details, consult the OpenAI API documentation linked from the course repository rather than relying on old examples found elsewhere on the Web.

API Credentials

You will use your own OpenAI API account for this homework. Start with approximately $10 of API credit or a comparable spending limit. You should not need to spend the full amount for this assignment. Configure a hard spending limit or usage alert in your API account if available, and monitor your usage while experimenting with the application.

Your ChatGPT subscription and the OpenAI API are separate services; a ChatGPT subscription does not by itself supply API credit.

Never place an API key directly in source code and never commit an API key to GitHub. The starter repository is configured so that credentials are obtained through Streamlit’s secrets mechanism.

If you accidentally commit a real credential to a repository, assume that it has been compromised and revoke it immediately.

Deploying ShopBot to the Cloud

You will deploy your ShopBot application so that it can be accessed through a normal Web browser. Streamlit includes its own web server that runs on a local port. Nevertheless, there is also an easy way to make your application available on the Internet through Streamlit Community Cloud, which can deploy a Streamlit application directly from a GitHub repository.

The basic procedure is:

1. Make sure the application runs locally and that `requirements.txt` contains its Python dependencies.
2. Copy the application to your own private repository (for which you will have admin privileges).
3. Sign in to Streamlit Community Cloud and connect the appropriate (i.e. your private, not the class homework') GitHub repository.
4. Create a new app and select the repository, branch, and `app.py` entry point.
5. Configure the API credential using Streamlit's secrets facility. Do *not* put the credential in the repository.
6. Deploy the application and verify that the assigned public URL loads correctly.

Streamlit Community Cloud redeploys the application when changes are pushed to the corresponding GitHub repository. Detailed screenshots are intentionally omitted because the service's interface may change; follow the current Streamlit deployment documentation if the interface differs from these instructions.

Remember that the deployed application is Internet-facing and intentionally vulnerable during part of this homework. It must contain only the synthetic data supplied for this assignment. Do not add real personal, account, payment, or university data to ShopBot.

Do not advertise or publicly distribute the URL of your deployed application. When you are not actively using it, you may take the application offline. If needed you can clamp it down further by adding network restrictions through Streamlit's web interface. (Such network restrictions are a useful way to reduce unintended use of your API account, but they are *not* a substitute for the application-level defenses required in this homework.) Monitor your API usage while the application is deployed to prevent any misuse.

Part I: Application Study (10 points)

Study the supplied application carefully before attempting to attack it. Describe its design by directly answering the following questions, focusing on architectural design rather than line-by-line code behavior.

Your response should identify:

1. the major components of the application;

2. what information is sent to the LLM;
3. what information is maintained as trusted application state;
4. how the LLM requests a purchase-history lookup;
5. where the database operation is performed; and
6. the important trust boundaries in the system.

Include a small architecture diagram. Your diagram need not be elaborate, but it should make clear which components exchange information and where you believe security decisions are being made.

Part II: Security Analysis (20 points)

The intended security policy is that a customer may obtain purchase-history information only for that customer's account.

Analyze the starter application and identify how this policy can be violated. Do not merely state that "prompt injection is possible." Identify the relevant data and control flow in the application.

Your response should address the following questions:

1. Under what conditions can the vulnerability be exploited?
2. What is the main reason for the vulnerability?
3. Which value or operation relevant to authorization is influenced by the LLM?
4. Which component currently appears to be trusted to enforce the policy?
5. What assumptions about LLM behavior must hold for the application to remain secure?

Keep your discussion at a design level. We are interested in the security architecture, not merely in the particular wording of any successful prompt.

Part III: Prompt-Injection Attacks (25 points)

Begin by finding at least one reasonably repeatable prompt-injection attack that causes ShopBot to violate the stated purchase-history security policy. The goal of this first step is to establish experimentally that the policy can be violated; you should not need to discover an especially obscure or elaborate prompt before proceeding.

Next, explore the attack surface more systematically. Construct at least three *meaningfully different* successful attacks rather than minor variations of the same string. Your attacks should represent different strategies for influencing the model. At minimum, your exploration should include:

1. a direct attempt to override or conflict with the application's instructions;
2. an attack that does not depend on obvious phrases such as "ignore previous instructions"; and
3. a third attack using a substantially different strategy, such as a multi-turn interaction, role/context manipulation, or another technique you discover experimentally.

You are encouraged to try additional strategies, including ones that fail. A failed attack can still be useful evidence about the behavior of the system, but it does not count toward the three successful attacks required above.

Record your attacks in the supplied `attacks.json` file. A template is included in the starter repository. **You may not modify the structure of the template**, except to add additional entries having the same structure (with a different `id`). Replace the placeholder content in each entry with your actual attack instructions. Do not add, remove, or rename fields in an attack entry; the provided `run_attacks.py` script and the instructor's evaluation infrastructure expect this format. Each attack entry must contain an identifier, a short strategy label, the simulated logged-in customer ID, and the sequence of user messages needed to reproduce the attack. For a multi-turn attack, place multiple user messages in the `messages` array; ShopBot's replies between turns are generated automatically when the attack is replayed.

You can replay the attacks in your file using:

```
python run_attacks.py attacks.json
```

The runner reports whether the model requested another customer's records and whether unauthorized records were actually returned. It also writes a detailed `attack-results.json` trace for your own analysis. This results file need not be submitted.

For each successful exploit, discuss in your report:

1. the identity of the simulated logged-in customer;
2. the information that customer is authorized to obtain;
3. the unauthorized information obtained;
4. a brief explanation of the strategy used and why you believe the attack succeeds; and
5. the observed reliability of the attack across repeated trials.

LLM behavior is not perfectly deterministic. An exploit need not succeed on every invocation, but it must be reasonably repeatable. If success varies, report this and describe how many trials you performed and how often the attack succeeded. Your objective is not to discover a single "magic phrase," but to understand how different forms of adversarial input can exercise the same underlying security weakness.

Part IV: Prompt-Level Defenses (15 points)

Attempt to defend ShopBot *without changing the application's authorization logic*. You may modify the system/developer instructions supplied to the model, add delimiters or explicit rules, perform input checks, or use another prompt-level/input-level technique that you think will make the attacks more difficult.

Test your modified application by rerunning the same `attacks.json` from Part III, and then try several variations of those attacks. Report which attacks are stopped and which still succeed. If all of your original attacks are stopped, actively attempt to construct new prompts that bypass your defense. You may add newly discovered attacks to `attacks.json`; keep the original attacks as well so that you can compare behavior before and after your defense.

Your written submission should answer:

1. What defense did you implement?
2. What threat did you expect it to address?
3. How did you test it?
4. Does the defense eliminate the underlying vulnerability, or does it primarily make exploitation more difficult? Explain.

Do not proceed on the assumption that a defense is strong simply because your first few attack strings fail.

Part V: Retrofitting a Robust Defense (20 points)

Now modify the application so that the purchase-history security policy is enforced by application code rather than by relying on the LLM to obey instructions.

Your final application must satisfy the following property:

> Regardless of the instructions supplied by the user or the tool invocation requested by the LLM, a customer must not be able to obtain another customer's purchase-history records through ShopBot.

You may implement any design that you believe enforces this property. We deliberately do not prescribe a particular code change. Your defense should be general: it should stop your submitted exploits and other similar attacks at the same security boundary.

Demonstrate at least the following tests:

- legitimate purchase-history queries for the logged-in customer continue to work;
- rerunning your submitted `attacks.json` produces no unauthorized retrievals;
- reasonable variations of those attacks also fail to disclose another customer's records; and

- ordinary, nonmalicious chatbot functionality remains usable.

It is acceptable for the LLM to *request* an unauthorized tool call in your final design, provided that application code prevents the unauthorized records from being returned. The important security property in this part is the behavior of the complete system, not whether the model itself can still be manipulated.

Describe the technique you used to retrofit the application and explain where authorization is enforced in your revised design.

Part VI: Security Analysis and Reflection (10 points)

Answer each of the following briefly but precisely.

1. Why is a stronger system prompt not equivalent to application-layer authorization?
2. In your final design, which component acts as the reference monitor for access to purchase-history records?
3. Does the LLM need to be trusted to preserve the purchase-history security policy in your final design? Explain.
4. Suppose an attacker could completely control the LLM's output, including every tool call it requests. What security property, if any, would your final application still guarantee? Explain why.

Submission Instructions

Submit the following artifacts. The exact electronic submission mechanism will be announced later.

- Your final source code.
- `attacks.json`, using the supplied format and containing the executable attack conversations from Part III.
- `report.pdf`, organized according to Parts I–VI above.
- The URL of your deployed ShopBot application.

Do not submit API keys, passwords, or other credentials. Before submission, verify that no credential has been committed anywhere in your repository history.

We will test your application using your own attacks and may also test it using similar attacks that you did not submit. Your final defense will therefore be evaluated on the security property it enforces, not only on whether it recognizes particular attack strings.

Collaboration and Academic Integrity

You may discuss general concepts, the design of LLM applications, Streamlit/API mechanics, and debugging strategies with other students. You may not exchange solutions, exploit strings, completed security analyses, or defense code. Also disclose uses of generative-AI tools as required by the GenAI policy stated earlier in this handout.

It is acceptable to submit an incomplete homework. It is not acceptable to obtain points by submitting another person's analysis, exploits, or implementation as your own.

Useful References

- PromptTrace: `https://prompttrace.airedlab.com/`
- Streamlit documentation: `https://docs.streamlit.io/`
- Streamlit Community Cloud deployment: `https://docs.streamlit.io/deploy/streamlit-communit`
- OpenAI API documentation: `https://developers.openai.com/api/docs/`
- OpenAI function/tool calling guide: `https://developers.openai.com/api/docs/guides/function-calling`